

《人工智能导论》实验二：情感分析报告

学号：2023010872

班级：计 37

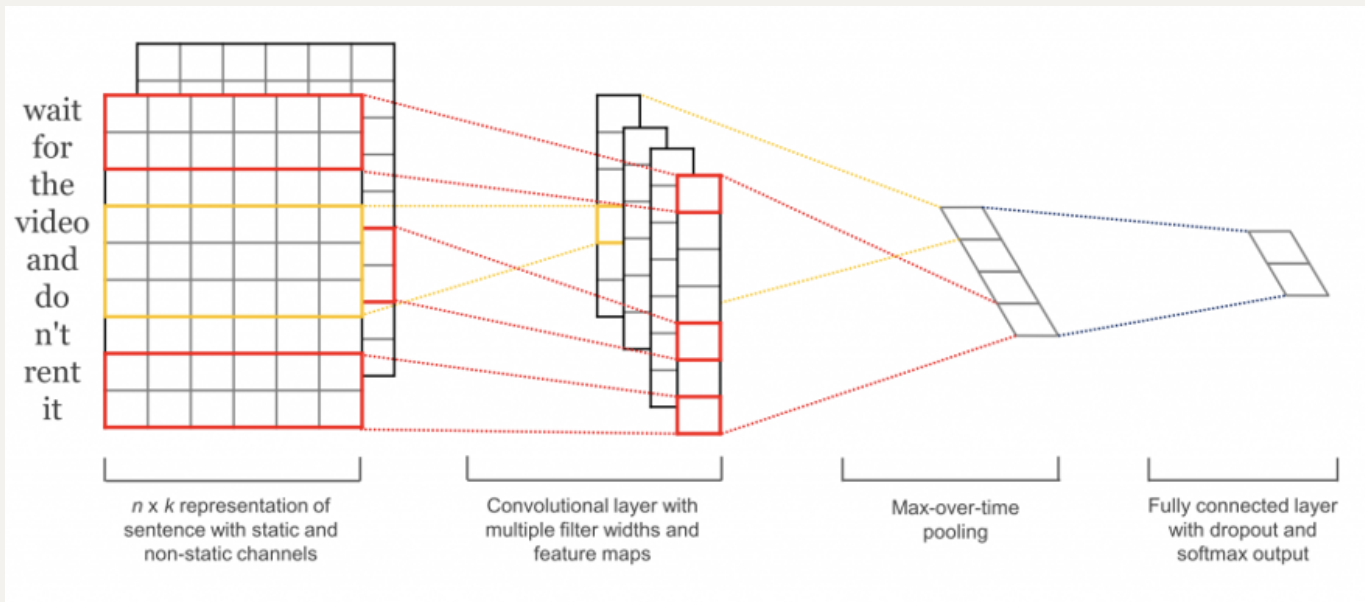
姓名：张馨元

一、模型结构图

本部分将展示实验中使用的卷积神经网络（CNN）和循环神经网络（RNN）的模型结构，并解释其关键组件的作用。

1.1 卷积神经网络 (CNN)

结构示意：



关键结构与作用解释：

基于 `src/model.py` 中的 `CNN` 类实现，其关键结构及作用如下：

1. 嵌入层 (Embedding Layer):

- **作用:** 将输入的文本序列（由词汇索引构成）映射为低维稠密的词向量表示。这是文本进入神经网络处理的第一步，使得离散的词语能被表示为连续的数值向量。
- **实现细节:** 可以加载预训练的词向量（本实验使用 Word2Vec），并根据 `freeze_embeddings` 参数决定是否在训练中微调词向量权重。使用 `PAD_IDX` 对变长序列进行填充，嵌入层会忽略填充部分的计算。

2. 卷积层 (Convolutional Layers) 与 ReLU 激活:

- **作用:** 通过在词向量序列上滑动不同尺寸的卷积核（filters），来捕捉文本中的局部模式，如关键的 n-gram 短语。不同尺寸的卷积核可以捕捉不同长度的短语特征。
- **实现细节:** 模型采用多个并行的 `nn.Conv2d` 层（通过将输入视为高度为 1 的图像实现一维卷积），每个卷积层使用不同尺寸的卷积核。卷积操作后，应用 ReLU (`F.relu`) 激活函数，引入非线性，增强模型的表达能力。

3. 池化层 (Pooling Layer):

- **作用:** 对卷积层输出的特征图进行降维操作，提取每个特征图中最显著的特征。这有助于减少参数数量，增强模型的泛化能力，并使得模型对特征的位置不那么敏感。
- **实现细节:** 对每个卷积核产生的特征图沿序列长度维度进行最大池化 (`F.max_pool1d`)。

4. 特征拼接 (Concatenation):

- **作用:** 将来自不同尺寸卷积核并通过池化层提取的特征向量拼接起来，形成一个更全面、更丰富的组合特征向量。
- **实现细节:** 使用 `torch.cat` 将所有池化后的向量在特征维度上进行拼接。

5. Dropout 层:

- **作用:** 在训练过程中以一定概率随机丢弃部分神经元的输出，以减少神经元之间的共同适应，从而防止模型过拟合，提高模型的泛化能力。
- **实现细节:** 在拼接后的特征向量输入到全连接层之前，应用 `nn.Dropout`。

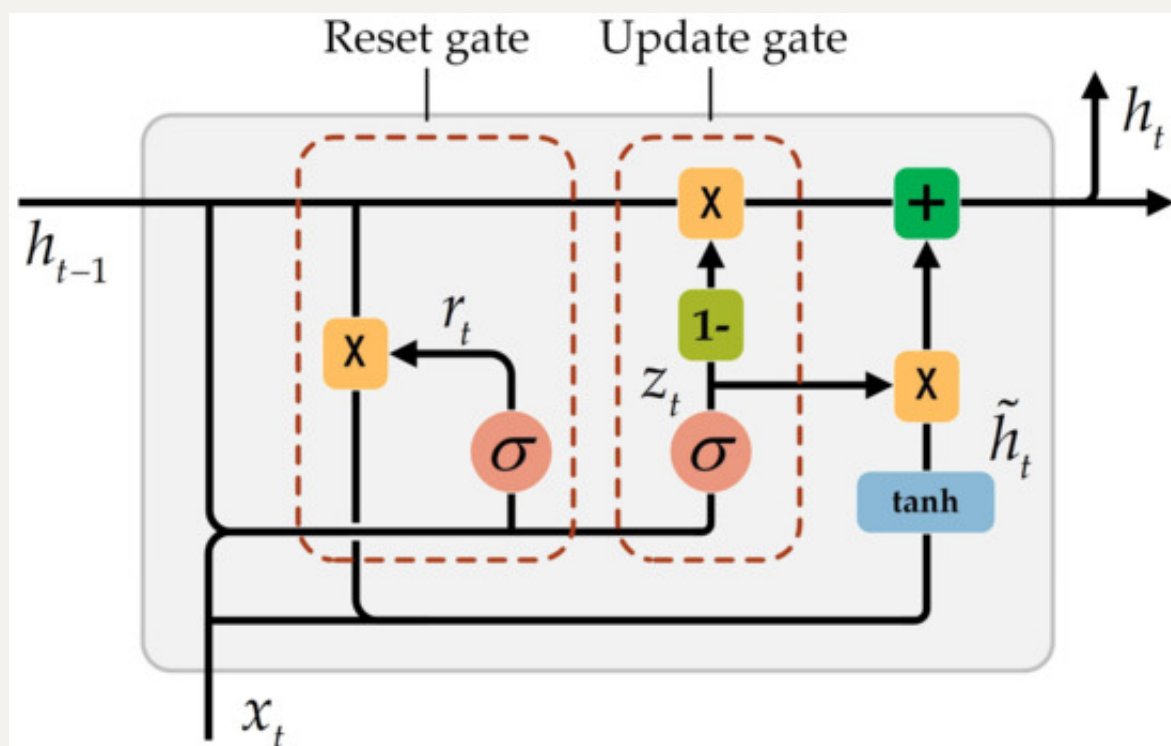
6. 全连接层 (Fully Connected Layer):

- **作用:** 将拼接后的高维组合特征映射到最终的输出维度。对于二分类情感分析任务，输出维度为 1，代表正向情感的 logit 值。

- 实现细节: 使用 `nn.Linear` 层进行映射。

1.2 循环神经网络 (RNN) - GRU

结构示意:



关键结构与作用解释:

基于 `src/model.py` 中的 `RNN` 类实现，使用门控循环单元（GRU），其关键结构及作用如下：

1. 嵌入层 (Embedding Layer):

- 作用: 与 CNN 中的嵌入层类似，将输入的词汇索引序列转换为词向量序列。
- 实现细节: 功能与 CNN 中的嵌入层一致。

2. GRU 层 (GRU Layer):

- 作用: 处理序列数据，通过其内部的门控机制（更新门和重置门）有效捕捉序列中的时间依赖关系和上下文信息，并缓解传统 RNN 中的梯度消失问题。
- 实现细节: 使用 `nn.GRU`。可以配置层数 (`num_layers`) 和是否双向 (`bidirectional`)。

- **多层 GRU:** 更深层次的抽象特征学习。
- **双向 GRU:** 同时从前向和后向处理序列，综合利用过去和未来的上下文信息。若使用，最后一个时间步的输出是前向和后向隐藏状态的拼接。
- **GRU 层输出所有时间步的隐藏状态以及最后一个时间步的（或多个最后时间步，取决于层数和方向）隐藏状态。**

3. 隐藏状态处理与 **Dropout**:

- **作用:** 从 GRU 层获取能代表整个序列信息的最终隐藏状态，并应用 Dropout 防止过拟合。
- **实现细节:** 对于分类任务，通常使用最后一个时间步的隐藏状态。如果是双向 GRU，则拼接最后一个时间步的前向和后向隐藏状态。然后应用 `nn.Dropout`。

4. 层归一化 (**Layer Normalization**):

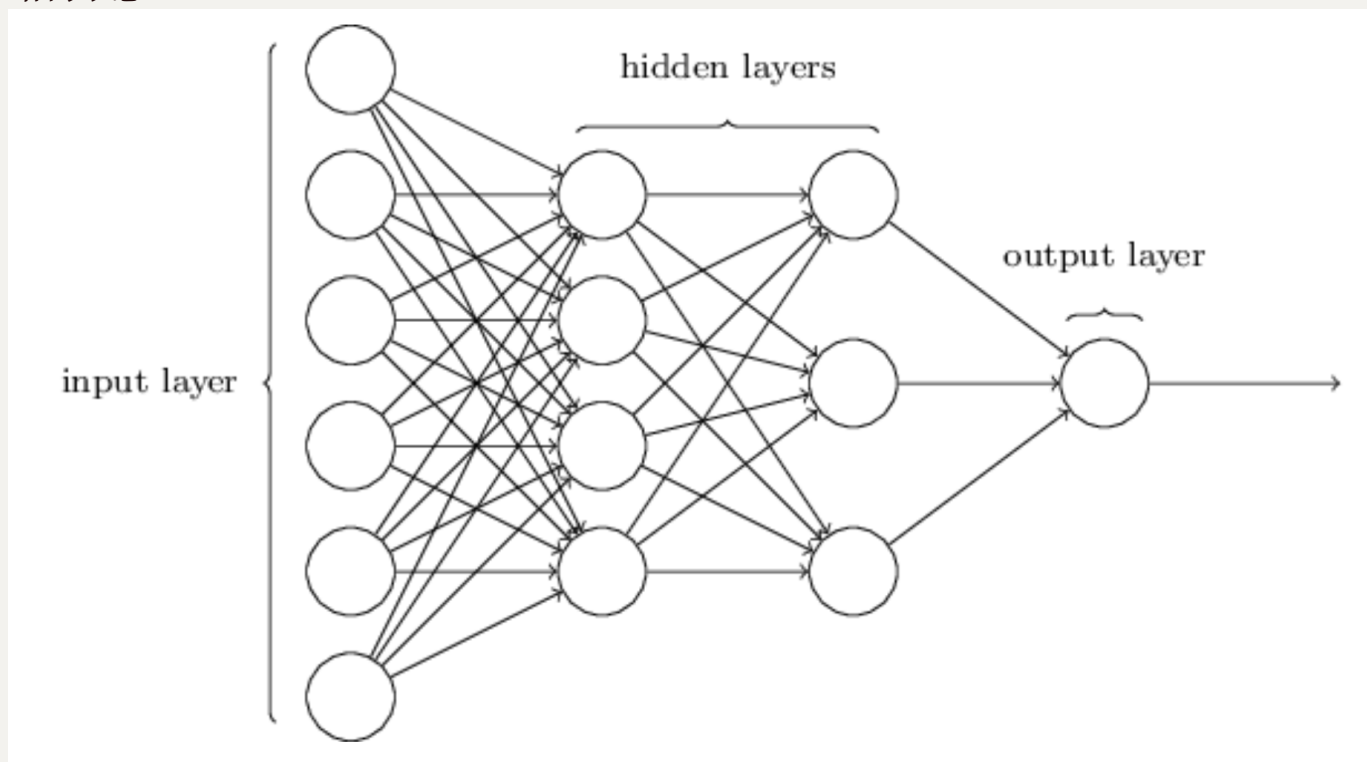
- **作用:** 对每个样本的特征进行归一化，有助于稳定 RNN 的训练过程，加速收敛，尤其在处理变长序列时。
- **实现细节:** 在 Dropout 之后、全连接层之前应用 `nn.LayerNorm`。

5. 全连接层 (**Fully Connected Layer**):

- **作用:** 将经过 GRU、Dropout 和层归一化处理后的序列表示映射到最终的输出维度 (logit 值)。
- **实现细节:** 使用 `nn.Linear` 层。

1.3 多层感知机 (MLP) - Baseline

结构示意：



关键结构与作用解释：

基于 `src/model.py` 中的 `MLP` 类实现，其关键结构及作用如下：

1. 嵌入层 (Embedding Layer):

- 作用: 与 CNN 和 RNN 中的嵌入层作用相同，将输入的词汇索引序列转换为词向量序列。
- 实现细节: 功能与前述模型的嵌入层一致。

2. 句子嵌入表示 (Sentence Embedding via Averaging Pooling):

- 作用: 将经过嵌入层转换后的变长词向量序列转换为一个固定长度的句子表示向量。这是因为标准 MLP 需要固定维度的输入。
- 实现细节: 通过对句子中所有非填充词 (non-PAD words) 的词向量进行平均 (mean pooling) 来实现。具体步骤包括:
 - i. 创建掩码 (mask) 以识别非填充词。
 - ii. 将填充词的词向量置零。
 - iii. 计算所有非填充词向量的总和。
 - iv. 将总和除以非填充词的数量，得到句子的平均嵌入向量。

3. 第一个全连接层 (Fully Connected Layer 1) 与 ReLU 激活、Dropout:

- 作用: 对平均句子嵌入向量进行第一次非线性变换, 提取更高层次的特征。
- 实现细节: 句子嵌入向量首先通过一个全连接层 (`self.fc1`), 将其映射到一个隐藏维度 (`hidden_dim`)。之后应用 ReLU 激活函数 (`F.relu()`) 引入非线性, 随后应用 Dropout (`self.dropout`) 以减少过拟合。

4. 第二个全连接层 (Output Layer / Fully Connected Layer 2):

- 作用: 将第一个隐藏层的输出映射到最终的分类输出维度。
- 实现细节: 经过 Dropout 处理的隐藏表示通过第二个全连接层 (`self.fc2`), 将其映射到最终的输出维度 (对于二分类问题为 1, 表示 logits 值)。

二、实验流程描述

本实验旨在对电影评论文本进行情感二分类。完整的实验流程包括数据加载与预处理、模型构建与初始化、模型训练, 以及最终的模型评估。

2.1 数据加载与预处理

1. 数据集描述:

- 实验数据集包含已预先分词的电影评论文本, 区分为训练集 (`train.txt`)、验证集 (`validation.txt`) 和测试集 (`test.txt`)。
- 提供了预训练的 Word2Vec 词向量 (`wiki_word2vec_50.bin`)。

2. 词表构建与索引化:

- 使用 `src/utils.py` 中的 `build_word2idx` 函数, 根据训练集 `train.txt` 构建词汇到索引的映射表 (`word2idx`)。词表外的词会被映射到一个特殊的 UNK (未知词) 索引。 `PAD_IDX` (通常为 0) 用于后续的序列填充。

3. 词向量加载与嵌入矩阵构建:

- 使用 `src/utils.py` 中的 `load_word2vec` 函数加载预训练的 `wiki_word2vec_50.bin` 词向量。

- 使用 `build_embedding` 函数，根据构建的 `word2idx` 和加载的预训练词向量，为当前词表创建一个嵌入矩阵。词表中的词如果存在于预训练向量中，则使用其向量；否则，可能使用随机初始化的向量（具体取决于 `build_embedding` 的实现，通常对未登录词随机初始化或置零）。

4. 数据封装 (`SentimentDataset`):

- 使用 `src/utils.py` 中的 `SentimentDataset` 类（继承自 `torch.utils.data.Dataset`）来封装数据集。
- 该类负责读取文本文件，将每个句子的词转换为索引序列，并根据设定的最大序列长度 `max_len` 进行截断或填充（使用 `PAD_IDX`）。每个样本包含填充/截断后的词索引序列及其情感标签。

5. 数据加载器 (`DataLoader`):

- 使用 `torch.utils.data.DataLoader` 为训练集、验证集和测试集创建数据加载器。
- `DataLoader` 负责按指定的 `batch_size` 生成数据批次，并在训练时对训练数据进行打乱 (`shuffle=True`)。

2.2 模型构建与初始化

1. 模型选择与实例化:

- 根据实验需求（通过命令行参数 `model_type` 指定），实例化 `src/model.py` 中定义的 `CNN`、`RNN (GRU)` 或 `MLP` 模型。
- 模型初始化时传入必要的参数，如词表大小 (`vocab_size`)、嵌入维度 (`embedding_dim`)、输出维度 (`output_dim=1`)、以及各模型特定的超参数（如 `cnn_num_filters`, `rnn_hidden_dim` 等）。
- 预训练的嵌入矩阵被传递给模型的嵌入层进行初始化，并根据 `freeze_embeddings` 参数决定是否在训练中冻结嵌入层权重。

2. 参数初始化策略:

- 嵌入层: 若提供预训练向量则使用之，否则 PyTorch 默认初始化。
- 其他层 (`nn.Linear`, `nn.Conv2d`, `nn.GRU`): 依赖 PyTorch 的默认初始化策略。例如，`nn.Linear` 和 `nn.Conv2d` 通常采用基于 Kaiming He 初始化（配合 ReLU）或 Xavier/Glorot 初始化的变体。

2.3 模型训练 (src/train.py)

1. 设备选择: 根据可用性选择在 CPU、CUDA (GPU) 或 MPS (Apple Silicon GPU) 上进行训练。
2. 优化器与损失函数:
 - 优化器 (**Optimizer**): 使用 Adam 优化器 (`optim.Adam`) 来更新模型参数, 学习率由 `learning_rate` 指定。
 - 损失函数 (**Criterion**): 使用二元交叉熵损失函数 `nn.BCEWithLogitsLoss`, 该函数内部包含了 Sigmoid 操作, 适用于二分类任务的 logits 输出。
3. 训练循环:
 - 进行指定 `num_epochs` 轮的训练。
 - 在每个 epoch 中:
 - 训练阶段:
 - 模型设置为训练模式 (`model.train()`)。
 - 遍历训练数据加载器 (`train_loader`) 产生的每个批次。
 - 将数据和标签移至选定设备。
 - 梯度清零 (`optimizer.zero_grad()`)。
 - 模型前向传播, 得到预测 logits。
 - 计算损失。
 - 反向传播计算梯度 (`loss.backward()`)。
 - 更新模型参数 (`optimizer.step()`)。
 - 累计训练损失和准确率。
 - 验证阶段:
 - 模型设置为评估模式 (`model.eval()`)。
 - 关闭梯度计算 (`with torch.no_grad()`)。
 - 遍历验证数据加载器 (`val_loader`) 产生的每个批次。
 - 模型前向传播, 得到预测 logits。
 - 计算验证损失。

- 累计验证损失和准确率。
 - 打印当前 epoch 的训练和验证结果。
4. 早停机制 (Early Stopping):
- 在每个 epoch 结束后，比较当前验证损失与之前的最小验证损失。
 - 如果验证损失在连续 `patience`（本实验中 `early_stopping_patience=3`）个 epoch 内没有改善（即没有降低），则提前终止训练，以防止过拟合。
 - 在训练过程中，我们会保存验证集损失最低时的模型参数作为最佳模型。
5. 返回最佳模型: 训练结束后，加载并返回在验证集上表现最佳的模型参数。

2.4 模型评估 (`src/evaluate.py`)

1. 加载最佳模型: 使用在训练过程中保存的最佳模型参数。
2. 评估模式: 模型设置为评估模式 (`model.eval()`)，并关闭梯度计算。
3. 测试集评估:
 - 遍历测试数据加载器 (`test_loader`)。
 - 模型前向传播，得到预测 logits。
 - 计算测试损失。
 - 将 logits 通过 Sigmoid 转换为概率，再根据 0.5 的阈值转换为类别预测 (0 或 1)。
 - 计算并报告在测试集上的准确率 (Accuracy)、精确率 (Precision)、召回率 (Recall) 和 F1 分数 (F1-score)。

三、实验结果展示

评价指标:

- 准确率 (Accuracy): `correct_predictions / total_predictions`
- F1-score: `2 * (Precision * Recall) / (Precision + Recall)`
- 精确率 (Precision): `True Positives / (True Positives + False Positives)`

- 召回率 (Recall): $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$

表 1: 不同模型性能对比

模型类型	ACCURACY	F1-SCORE	PRECISION	RECALL
CNN	0.8374	0.83871	0.8432	0.8342
RNN	0.8266	0.8232	0.8514	0.7968
MLP	0.7697	0.8387	0.8432	0.8342

上面的数据统一选用超参如下:

- learning rate: 0.0001*
- batch size: 64*
- 5 个 epoch 没有提升则早停(最多 100)*

四、参数对比分析

本实验中使用的主要超参数已在 `src/main.py` 的 `CONFIG` 字典中预设，并通过命令行参数提供调整的选项。

4.1 固定参数

通用参数

- `dropout: 0.5`
 - 防止过拟合。值过大可能导致欠拟合，过小则正则化效果不足。0.5 是常用值。
- `freeze_embeddings: True`
 - 是否冻结预训练词向量的权重。冻结可以减少训练参数，加速训练，并保留预训练向量的通用语义信息，适用于目标任务数据量不大时。若为 `False`，词向量将随模型一同微调，可能更适应特定任务，但数据量少时易过拟合。

- `max_len: 128`
 - 序列最大长度。用于文本截断和填充。应根据数据集中文本长度分布选择。

CNN 特定参数

- `cnn_num_filters: 100`
 - 每种尺寸卷积核的数量。增加此值可以学习更多特征，但也增加模型复杂度。
- `cnn_filter_sizes: [3, 4, 5]`
 - 卷积核的尺寸列表。例如，尺寸 3 的卷积核捕捉 3-gram 特征。组合不同尺寸可以捕捉不同长度的局部模式。

RNN (GRU) 特定参数

- `rnn_hidden_dim: 128`
 - GRU 隐藏层维度。增加此值可以增强模型记忆和表达能力，但也增加计算量和过拟合风险。
- `rnn_num_layers: 2`
 - GRU 的层数。多层 RNN 可以学习更抽象的序列表示。
- `rnn_bidirectional: True`
 - 是否使用双向 GRU。双向能同时考虑上下文，通常性能更好，但计算量加倍。

MLP 特定参数

- `mlp_hidden_dim: 64`
 - MLP 第一个全连接层后的隐藏维度。

参数对比

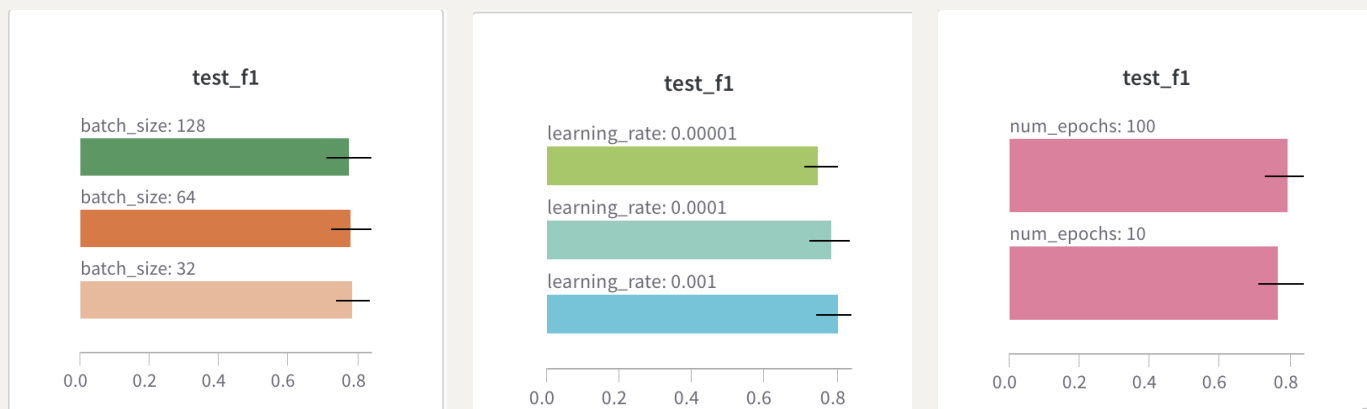
我主要对 learning rate, batch size 和 num_epochs 进行了测试，即：

```
#!/bin/bash

for model_type in cnn rnn mlp; do
    for epochs in 10 100; do
        for batch_size in 32 64 128; do
            for lr in 0.001 0.0001 0.00001; do
                bash main.sh $model_type $epochs $batch_size $lr
            done
        done
    done
done
```

其中,我设置了 5 个 epoch 没有提高则早停，实测不会有超过 100 个 epoch 还没有训练结束的，所以 epochs 100 实际使用早停逻辑。

实验结果如下：(针对 F1-Score 指标进行分析)



从左到右分别对应batch size, learning rate 和 epochs(早停与否)的实验结果

黑线代表最大值和最小值

我们有如下发现：

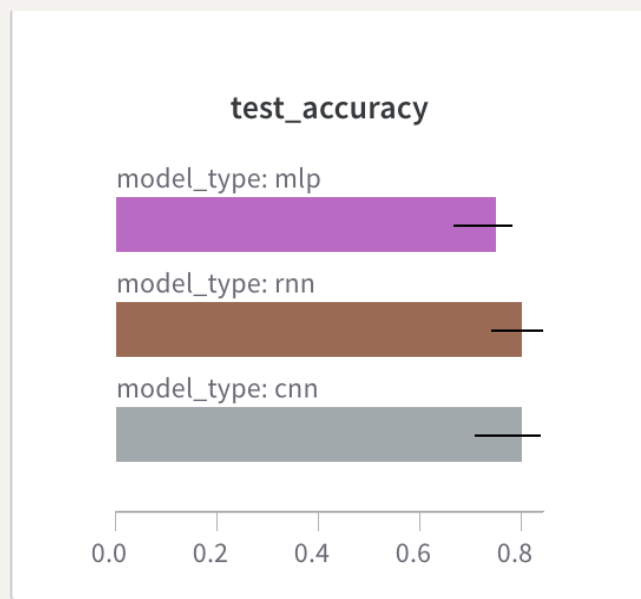
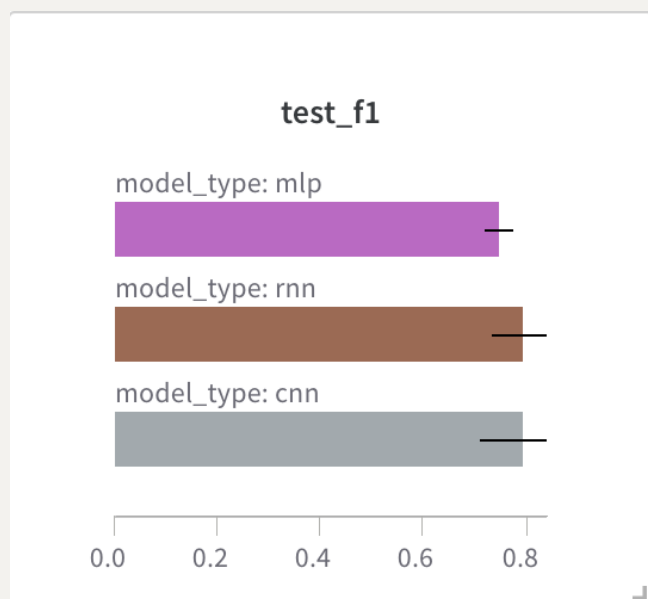
1. 总体而言，由于该任务相对简单且预训练向量已给定，故参数影响不大。
2. 参数在平均意义上的最好和最大意义上的最好是不同的。以 batch size 为例，32 的平均值最高，但是 64 的最大值最高。

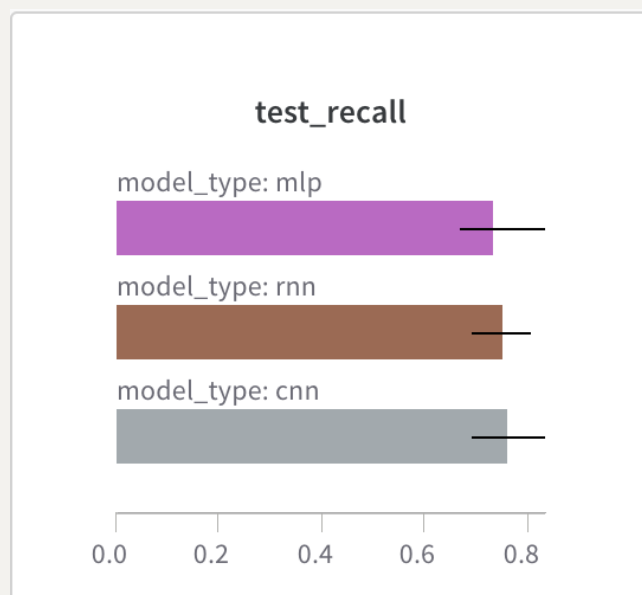
3. batch size 小则模型结果分布更稳定（极差更小）
4. learning rate 越大效果越好，或许是任务简单，因此不容易陷入局部最优，且较大的学习率可以帮助模型更快地跳出次优解，加速收敛。
5. 早停效果好于限制 10 个 epoch，说明模型在训练初期就能较快达到较好性能，额外的训练轮次可能带来的提升有限，而早停机制能够有效避免过拟合，保持模型的泛化能力。

五、模型比较

本部分对比实验中实现的 CNN、RNN (GRU) 以及基线模型 MLP 在情感分析任务上的效果和特性。

5.1 性能对比 (基于预期和典型表现)



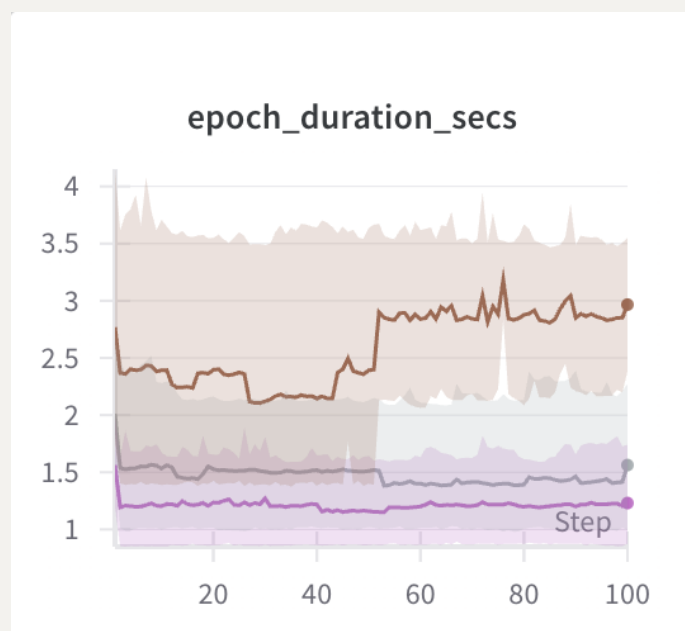


从上到下分别对应 MLP, CNN 和 RNN 的实验结果

根据上述实验结果图表和第三部分的性能指标，我们可以对三种模型的性能进行如下分析：

MLP 显著比 CNN 和 RNN 效果差，但是 CNN 和 RNN 效果相差不大，在不同的指标上各有优劣。

训练时间 (使用单卡 NVIDIA GeForce RTX 4090)：



从上到下分别对应 RNN, CNN 和 MLP 的实验结果

可见，RNN训练时间显著更长，MLP 训练时间最短。

5.2 各自优缺点总结

1. 全连接神经网络 (MLP / Feedforward Neural Network):

- 优点:
 - 结构简单，易于理解和实现。
 - 通用性强: 理论上可以拟合任何复杂的函数。
 - 计算相对高效: 如果网络层数和节点数不是特别巨大。
- 缺点:
 - 参数量大: 如果输入维度很高，全连接层参数会非常多，容易过拟合。
 - 缺乏对序列顺序或空间结构的利用: 对文本，通常将变长序列转换为固定长度向量（如平均词向量），丢失词序和局部上下文。
 - 对输入维度敏感。

2. 卷积神经网络 (CNN):

- 优点:
 - 参数共享: 大大减少模型参数量，降低过拟合风险。
 - 局部连接: 能学习局部特征（如文本中的 n-gram）。
 - 平移不变性 (通过池化层): 对特征位置不敏感。
 - 层次化特征学习。
 - 在文本分类中能有效捕捉关键局部短语。
- 缺点:
 - 对长距离依赖关系捕捉能力有限。
 - 对词序的敏感性不如 RNN。

3. 循环神经网络 (RNN - 特指 GRU):

- 优点:
 - 擅长处理序列数据: 捕捉序列中的时间依赖和上下文。
 - 记忆能力 (GRU/LSTM): 通过门控机制学习长期依赖。
 - 对词序敏感。
 - 参数共享 (跨时间步)。

- 缺点：
 - 训练速度慢: 序列依赖性导致难以并行。
 - 梯度消失/爆炸问题 (GRU/LSTM 缓解但仍可能存在于极长序列)。
 - 标准 RNN 仅单向上下文 (双向可解决但增加复杂度)。

六、问题思考回答

问题 1: 实验训练什么时候停止是最合适的? 简要陈述你的实现方式, 并试分析固定迭代次数与通过验证集调整等方法的优缺点。

实现方式:

在本次实验的代码 (`src/train.py`) 中, 我们采用了 早停 (**Early Stopping**) 机制来决定训练的停止时机。具体实现如下:

- 在每个 epoch 训练完成后, 在验证集上评估模型的性能 (以验证集损失 `val_loss` 为主要指标)。
- 如果验证集损失在连续 `patience` (本实验中 `early_stopping_patience=5`) 个 epoch 内没有得到改善 (即没有降低), 则提前终止训练。
- 在训练过程中, 我们会保存验证集损失最低时的模型参数作为最佳模型。

固定迭代次数的优缺点:

- 优点:
 - 实现简单, 易于控制训练总时长。
 - 对于某些已知收敛特性的任务或模型, 可以预估一个合适的迭代次数。
- 缺点:
 - 可能欠拟合: 如果迭代次数设置过少, 模型可能没有充分学习数据中的模式, 导致性能不佳。
 - 可能过拟合: 如果迭代次数设置过多, 模型可能会在训练数据上过度拟合, 学习到噪声, 导致在未见过的验证集或测试集上性能下降。
 - 难以找到一个通用的"最佳"迭代次数, 它往往依赖于具体数据集、模型结构和超参数。

通过验证集调整（如早停）的优缺点：

- 优点：
 - 防止过拟合：通过监控验证集性能，可以在模型开始过拟合（表现为验证集性能不再提升或开始下降）时及时停止训练。
 - 自动确定训练轮次：无需手动设置固定的迭代次数，模型会自动找到一个较优的训练点。
 - 通常能获得更好的泛化性能：因为模型是在验证集性能最优时停止的。
- 缺点：
 - 对验证集质量敏感：如果验证集不能很好地代表测试数据的分布，或者验证集过小，早停可能不会带来预期的效果，甚至可能提前停止在次优点。
 - 增加了监控开销：需要在每个 epoch 后进行验证集评估，增加了训练时间。
 - **Patience** 参数的选择：`patience` 参数本身也需要调整，设置不当可能导致过早停止或训练时间过长。

总的来说，通过验证集调整（尤其是早停机制）是更常用且通常更有效的方法，因为它能更好地平衡欠拟合和过拟合，并倾向于找到泛化能力更强的模型。

问题 2: 实验参数的初始化是怎么做的？不同的方法适合哪些地方？

在我们的实验代码 (`src/model.py`) 中，参数初始化主要依赖于 PyTorch 各层（如 `nn.Embedding`, `nn.Linear`, `nn.Conv2d`, `nn.GRU`）的默认初始化策略，同时也提供了加载预训练词向量的选项。

- **`nn.Embedding`**：
 - 如果提供了 `pretrained_embedding`（如本实验中的 Word2Vec 向量），则使用预训练的词向量进行初始化。这是一种非常有效的方法，因为它利用了大规模文本语料库预先学习到的语义信息。
 - 如果没有提供预训练向量，`nn.Embedding` 默认使用均值为 0，标准差为 1 的正态分布 ($N(0,1)$) 进行初始化。
- **`nn.Linear` 和 `nn.Conv2d`**：

- PyTorch 的这些层通常采用基于 Kaiming He 初始化（针对 ReLU 激活函数）或 Xavier/Glorot 初始化（针对 Sigmoid/Tanh 激活函数）的变体。这些方法旨在使每一层的输出方差和梯度的方差保持在合理的范围内，从而加速收敛并避免梯度消失/爆炸问题。
- 例如，`nn.Linear` 的权重默认使用均匀分布 $U(-\sqrt{k}, \sqrt{k})$ 初始化，其中 $k = 1 / \text{in_features}$ 。
- **nn.GRU:**
 - GRU（以及 LSTM）的权重和偏置也有其默认的初始化方式，通常是均匀分布或正态分布，旨在维持较好的梯度流。

不同的初始化方法及其适用场景：

1. 零初始化 (Zero Initialization):

- 缺点: 导致对称性问题，模型无法学习。通常不用于权重，可用于部分偏置项。
- 适用: 特定情况下的偏置项。

2. 随机初始化 (Small Random Values):

- 优点: 打破对称性。
- 缺点: 值过大或过小都可能导致梯度问题。
- 适用: 简单网络，但常被更高级方法取代。

3. Xavier/Glorot 初始化:

- 旨在使每层激活值和梯度的方差保持一致。
- 适用: 常用于 Sigmoid 和 Tanh 激活函数。

4. Kaiming He 初始化:

- Xavier 的改进版，特别适用于 ReLU 及其变体。
- 适用: ReLU、ReLU 及其变种激活函数。是 CNN 中常用的初始化方法。

5. 正交初始化 (Orthogonal Initialization):

- 优点: 有助于保持梯度范数，尤其在 RNN 中缓解梯度消失/爆炸。
- 适用: 常用于 RNN 的权重初始化。

6. 使用预训练权重 (Pre-trained Weights):

- 如实验中嵌入层使用预训练词向量。

- **优点:** 显著加速收敛，通常能达到更好的性能，尤其在目标任务数据量较少时（迁移学习）。
- **适用:** 几乎所有类型的模型，只要有合适的预训练模型可用。嵌入层使用预训练词向量是 NLP 任务中的标准做法。

选择哪种初始化方法取决于网络结构、激活函数类型以及是否有可用的预训练模型。PyTorch 的默认初始化通常是一个不错的起点。

问题 3: 过拟合是深度学习常见的问题，有什么方法可以防止训练过程陷入过拟合？

过拟合是指模型在训练数据上表现良好，但在未见过的测试数据或验证数据上表现较差的现象。防止过拟合的方法主要有：

1. 获取更多数据 (Data Augmentation):

- 最有效的方法之一。对于文本，可使用回译、同义词替换、随机插入/删除/交换等。

2. 正则化 (Regularization):

- 通过向损失函数添加惩罚项来限制模型复杂度。
- **L1 正则化 (Lasso):** 产生稀疏权重，可用于特征选择。
- **L2 正则化 (Ridge / Weight Decay):** 使权重值变小。实验代码中的优化器通常可以设置 `weight_decay` 参数即为 L2 正则化。
- **Elastic Net:** L1 和 L2 的结合。

3. Dropout:

- 在训练中随机丢弃神经元。迫使网络学习更鲁棒的特征。本实验中 CNN、RNN 和 MLP 模型均使用了 Dropout。

4. 早停 (Early Stopping):

- 在验证集性能不再提升时停止训练。本实验已采用。

5. 选择合适的模型复杂度 (Model Simplification):

- 减少网络层数、神经元数量、滤波器数量等。

6. 批归一化 (Batch Normalization) 和 层归一化 (Layer Normalization):

- **Batch Normalization (BN):** 对小批数据归一化，加速训练，有正则化效果。MLP 模型中可考虑使用。

- **Layer Normalization (LN):** 对单个样本特征归一化。RNN 中常用。本实验 RNN 模型中已加入。

事实上，在较为现代的模型中，BN 和 LN 已经成为了标配，并取代了复杂的初始化方法。

七、心得体会

通过本次《人工智能导论》的实验二，我对基于深度学习的情感分析任务有了更为系统和深入的理解。主要心得体会如下：

1. 模型结构与特性理解：

- 亲手实现和调试 CNN、RNN (GRU) 以及 MLP 模型，使我不再仅仅停留在理论层面，而是具体地观察到它们如何处理文本数据。例如，CNN 通过不同尺寸的卷积核捕捉局部短语特征，而 GRU 则通过其循环结构和门控机制来处理序列依赖关系。MLP 作为基线，则凸显了序列信息对于情感分析的重要性。
- 理解了嵌入层 (Embedding Layer) 的核心作用——将离散的词汇索引映射到低维稠密的向量空间，以及加载预训练词向量（如 Word2Vec）对于提升模型性能和加速收敛的巨大价值。`freeze_embeddings` 参数的设置也让我思考了在不同数据规模下是否微调词向量的权衡。

2. 实验流程与参数调优：

- 熟悉了从数据预处理（分词已提供，主要是构建词表、序列填充）、模型构建、定义损失函数与优化器、模型训练、验证集评估到最终测试集评估的完整深度学习实验流程。
- 参数调优是深度学习实验中至关重要的一环。在本次实验（或未来可能的探索）中，学习率 (`learning_rate`)、批大小 (`batch_size`)、Dropout 率 (`dropout`)、CNN 的滤波器数量和尺寸 (`cnn_num_filters`, `cnn_filter_sizes`)、RNN 的隐藏单元数和层数 (`rnn_hidden_dim`, `rnn_num_layers`) 等超参数，都需要细致调整。例如，过大的学习率可能导致模型不收敛，过小的学习率则收敛缓慢；合适的 Dropout 率是防止过拟合的关键。
- 早停 (Early Stopping) 机制的引入非常实用，它能有效防止模型在训练集上过拟合，并自动确定一个较优的训练轮次，节省了大量手动调整 `num_epochs` 的时间。

3. 代码实现与调试：

- 在 PyTorch 框架下进行模型搭建和训练，进一步熟练了其常用模块如 `nn.Module`, `nn.Linear`, `nn.Conv2d`, `nn.GRU`, `nn.Embedding`, 优化器 (`torch.optim`) 和损失函数 (`nn.BCEWithLogitsLoss`) 的使用。
- 调试过程中，理解张量 (Tensor) 的维度变化至关重要。尤其是在拼接不同卷积核的输出或处理双向 RNN 的隐藏状态时，需要确保维度匹配，否则容易出错。
- 代码的模块化和清晰结构（如将模型定义、训练循环、评估函数、工具函数分别放在不同 `.py` 文件中）对于项目的可维护性和可读性非常有益。

4. 对情感分析任务的思考：

- 情感分析不仅仅是简单地判断文本的褒贬，它涉及到对语言细微差别的理解，如否定、转折、讽刺等。虽然本次实验的模型取得了一定的效果，但更复杂的语言现象可能需要更先进的模型（如 Transformer）或更精细的特征工程。
- 预训练模型的强大之处在于它们从大规模无标签数据中学习到的通用语言表示，这为许多下游 NLP 任务提供了一个坚实的基础。

5. 反思与展望：

- 本次实验中，如果时间允许，可以尝试更多的超参数组合，或者探索其他正则化技巧。
- 对于模型的可解释性，例如 CNN 学到了哪些关键短语，RNN 如何利用上下文，也是一个值得进一步研究的方向。
- 未来可以尝试更复杂的模型架构，比如带有注意力机制的 RNN，或者直接使用预训练的 Transformer 模型（如 BERT）进行微调，以期获得更好的性能。

总的来说，这次实验是一次宝贵的实践机会，不仅巩固了理论知识，也提升了动手能力和问题解决能力。我对深度学习在自然语言处理领域的应用有了更深的体会和兴趣。